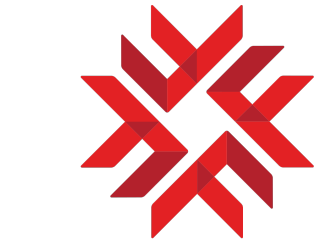
Tensorflow II

Two-Stage Object Detectors

Fatemeh Dehrouyeh



Contents

1. What object detection must do
2. The Original Region-Based Idea
3. R-CNN
4. Fast R-CNN
5. Faster R-CNN
   - Lab
6. Mask R-CNN
   - Lab

Current Section

1 What object detection must do

2 The Original Region-Based Idea

3 R-CNN

4 Fast R-CNN

5 Faster R-CNN

6 Mask R-CNN

What Must an Object Detector Do?

Object detection is more than image classification.

A detector must answer three questions:

1. **What object is present?**
2. **Where is the object located?**
3. **How confident is the model?**

Output

An object detector produces:

$$\text{Class Label} + \text{Bounding Box} + \text{Confidence Score}$$

Detection Output Example

For one detected object, the model may output:

$$\text{person}, \quad [x_{\min}, y_{\min}, x_{\max}, y_{\max}], \quad 0.94$$

This means:

- **Class:** person
- **Bounding box:** object location in the image
- **Confidence score:** how certain the model is

Key Idea

Object detection combines classification and localization.

From Classification to Detection

Image classification gives one label for the whole image.

$$\text{Image} \rightarrow \text{Class Label}$$

Object detection gives labels and locations for objects.

$$\text{Image} \rightarrow \text{Boxes} + \text{Classes} + \text{Scores}$$

Main Difference

Classification answers: **What is in the image?**
Detection answers: **What is in the image, and where is it?**

Many Objects, Many Predictions

A single image can contain many objects.

For example, a street image may contain:

- cars,
- pedestrians,
- traffic lights,
- bicycles.

The detector must produce one prediction for each object:

$$(\text{box}_1, \text{class}_1, \text{score}_1), (\text{box}_2, \text{class}_2, \text{score}_2), \ldots$$

Challenge

The model must decide both **how many objects** exist and **where they are**.

Reminder: IoU and NMS

Before studying detector architectures, recall two important ideas.

Intersection over Union

IoU measures how much two bounding boxes overlap.
It is used to compare predicted boxes with ground-truth boxes.

Non-Maximum Suppression

NMS removes duplicate detections.
It keeps the highest-confidence box and suppresses highly overlapping boxes.

Why This Matters

Detector architectures produce many candidate boxes. IoU and NMS help decide which boxes are useful.

Main Question

Now that we know what the detector must output, we can study how different architectures produce those outputs.

How does a model produce object classes and bounding boxes from an image?

Next

We first compare **one-stage** and **two-stage** object detectors.

Current Section

1 What object detection must do

2 The Original Region-Based Idea

3 R-CNN

4 Fast R-CNN

5 Faster R-CNN

6 Mask R-CNN

Why Region-Based Detection?

A classification CNN can answer:

What object is in this image?

But object detection must also answer:

Where is each object?

Early region-based detectors solved this by separating the problem into two parts:

Find possible object regions $\rightarrow$ Classify those regions

Key Idea

Instead of testing every possible rectangle in an image, generate a smaller set of promising candidate regions first.

What Is a Region Proposal?

Before classifying an object, some detectors first identify areas of the image that are likely to contain objects.

These areas are called:

Region Proposals or Candidate Regions

For example, instead of examining the entire image as one object, the detector may propose several regions:

- a region that may contain a person,
- a region that may contain a car,
- a region that may contain another object.

Key Idea

A region proposal does not yet say **what** the object is.
It only says: *"There may be an object here."*

Current Section

1 What object detection must do

2 The Original Region-Based Idea

3 R-CNN

4 Fast R-CNN

5 Faster R-CNN

6 Mask R-CNN

R-CNN: The Basic Idea

R-CNN stands for **Regions with CNN Features**.

Its detection pipeline is:

Image → Region Proposals → CNN Features → Class Prediction → Box Refinement

For one image, the original R-CNN typically generated around **2,000 candidate regions**.
Each proposed region was then processed separately by a CNN.

Important

R-CNN did not learn where to propose regions.
The proposals came from an external algorithm called **Selective Search**.

How Selective Search Generates Regions

Selective Search does **not** begin with predefined boxes.
It begins by dividing the image into many small image regions.

$$\text{Image} \rightarrow \text{Initial Segmentation} \rightarrow \text{Many Small Regions}$$

Then neighbouring regions are compared using visual similarity.
Important cues include:

- colour similarity,
- texture similarity,
- region size,
- how well two regions fit together.

Selective Search: Hierarchical Merging

After the initial segmentation:

1. Compare neighbouring regions.
2. Find the most similar pair.
3. Merge those two regions.
4. Recalculate similarities with neighbouring regions.
5. Repeat the process until the entire image forms one region.

This creates a hierarchy:

$$\text{Small Regions} \rightarrow \text{Larger Regions} \rightarrow \text{Still Larger Regions}$$

Bounding boxes around regions at different levels become **candidate object proposals**. Selective Search ranks regions by generally favouring those formed later in the merging process.

Why Use Many Levels?

Small objects may appear at lower levels of the hierarchy, while larger objects may appear after several regions are merged.

Is Selective Search Trained?

No, not in the classical R-CNN pipeline.

Selective Search is an algorithmic region-proposal method.
It does not learn object classes from labeled training data.

It uses image properties such as:

- colour,
- texture,
- size,
- spatial compatibility.

Important Distinction

Selective Search generates possible object locations.
The **CNN and classifiers** are the learned parts of R-CNN.

How Was R-CNN Trained?

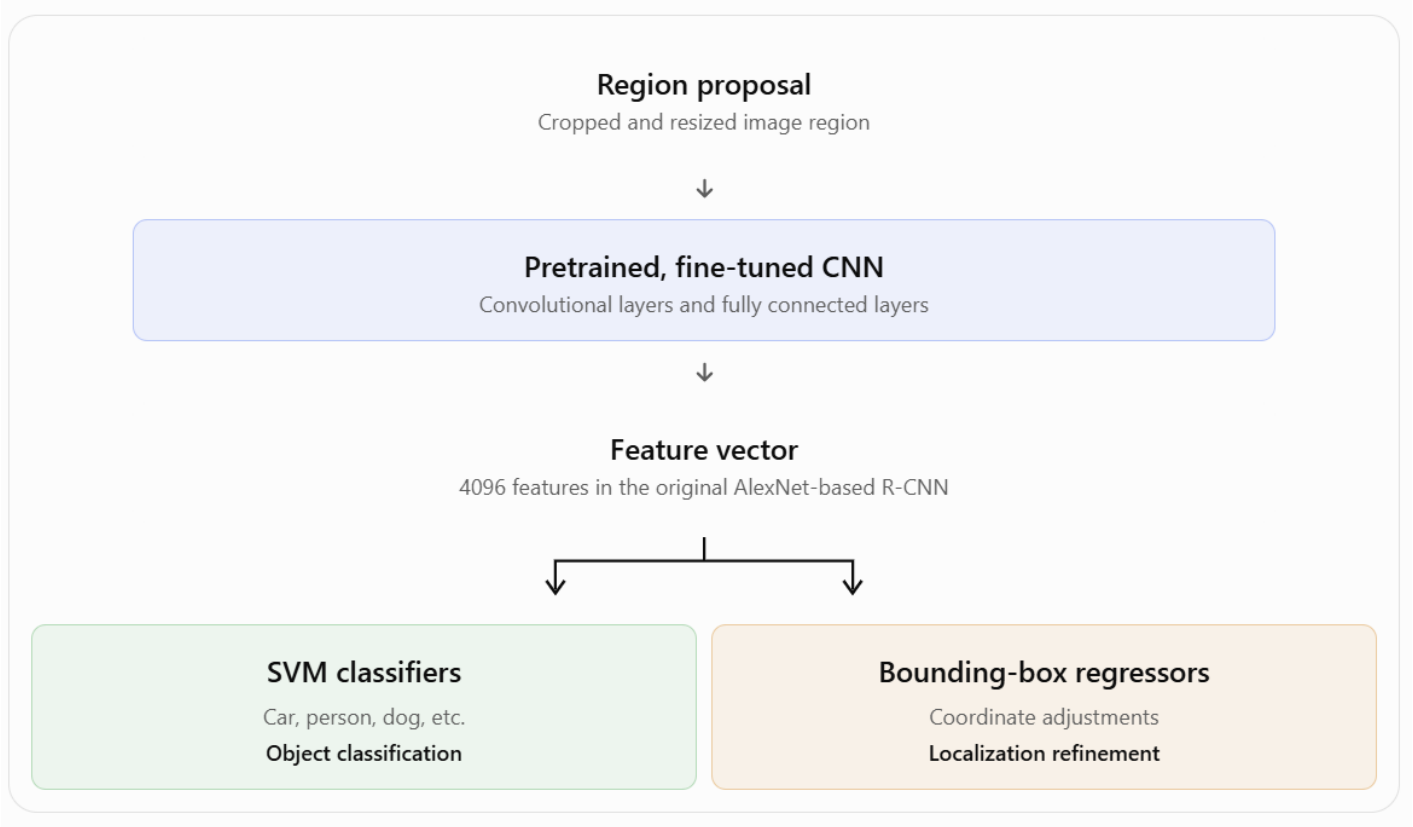
The original R-CNN training pipeline had several separate stages.

1. Start with a CNN pretrained for image classification (typically on ImageNet).
2. Fine-tune the CNN using region proposals labeled with object classes or background.
3. Remove the softmax classification layer and extract features from the last fully connected hidden layer.
4. Train a separate SVM classifier for each object class using the extracted CNN features.
5. Train bounding-box regressors to refine the predicted object locations.

Why Pretraining?

Object-detection datasets were relatively small.
The CNN first learned general visual features from a larger classification dataset, then adapted those features to detection.

Original R-CNN Architecture



How Does R-CNN Know Which Proposal Is Correct?

During training, every image contains ground-truth boxes.
Each region proposal is compared with the ground-truth boxes using IoU.

$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

A proposal with sufficient overlap can be treated as a **positive example** for that object class.
A proposal with little overlap is treated as **background**.

Learning

The classifier learns which visual features correspond to each class.
The bounding-box regressor learns how to move and resize a proposal so it better matches the true object. The regressor learns patterns in CNN features to predict box position and size adjustments, rather than absolute coordinates.

What Happens to Each Region Proposal?

Each candidate region is cropped from the image and resized to the input size expected by the CNN.

$$\boxed{\text{Region} \rightarrow \text{Resize} \rightarrow \text{CNN} \rightarrow \text{Feature Vector}}$$

The feature vector is then used for:

- object-class prediction,
- background prediction,
- bounding-box refinement.

Important

The CNN is run separately for many proposed regions.
This is one of the main reasons original R-CNN is slow.

R-CNN During Inference

For a new image:

1. Selective Search generates candidate regions.
2. Each region is cropped and resized.
3. The CNN extracts features from each region.
4. Classifiers assign object scores.
5. Bounding-box regression adjusts the boxes.
6. Low-confidence predictions are removed.
7. NMS removes duplicate detections.

$$\boxed{\text{Image} \rightarrow \text{Proposals} \rightarrow \text{CNN} \rightarrow \text{Classes + Refined Boxes} \rightarrow \text{NMS}}$$

Why Was Original R-CNN Replaced?

Original R-CNN was important, but inefficient.

Main problems:

- around thousands of region proposals per image,
- a separate CNN forward pass for each proposal,
- several separate training stages,
- extracted features were stored on disk so the separate SVM classifiers and bounding-box regressors can be trained later,
- slow training and slow inference.

Main Problem

The CNN repeatedly processes overlapping parts of the same image.
Later models reuse shared image features instead.

Is Original R-CNN Still Used Today?

Original R-CNN is mainly of historical and educational importance.

Modern libraries generally do not provide original R-CNN as a standard pretrained detector.
Instead, its descendants are widely available.

Examples include pretrained:

- Faster R-CNN,
- Mask R-CNN,
- SSD,
- RetinaNet,
- FCOS.

Practical Recommendation

Learn original R-CNN to understand the region-based idea.
For real applications, use a newer detector such as Faster R-CNN rather than original R-CNN.

From R-CNN to Faster Region-Based Detection

R-CNN established an important idea:

$$\boxed{\text{Propose Regions} \rightarrow \text{Classify and Refine Them}}$$

But its implementation was slow because each proposal was processed independently.

This led to a sequence of improvements:

$$\boxed{\text{R-CNN} \rightarrow \text{Fast R-CNN} \rightarrow \text{Faster R-CNN}}$$

Next

The next question is:
How did each generation make region-based detection faster?

Current Section

1 What object detection must do

2 The Original Region-Based Idea

3 R-CNN

4 Fast R-CNN

5 Faster R-CNN

6 Mask R-CNN

Why Did R-CNN Need to Evolve?

R-CNN introduced the region-based detection idea:

$$\text{Region Proposals} \rightarrow \text{CNN Features} \rightarrow \text{Classification + Box Refinement}$$

However, the original R-CNN had an important inefficiency:

- thousands of regions were proposed,
- each region was cropped separately,
- the CNN was run separately for each region,
- many overlapping regions repeated almost the same computation.

Main Problem

The same image was processed by the CNN many times.

This motivated the next model:

Fast R-CNN

Fast R-CNN: The Main Improvement

Fast R-CNN changed one major idea:

$$\boxed{\text{Run the CNN once on the whole image}}$$

instead of running the CNN separately for every region proposal.

The new pipeline became:

$$\text{Image} \rightarrow \text{CNN} \rightarrow \text{Feature Map}$$

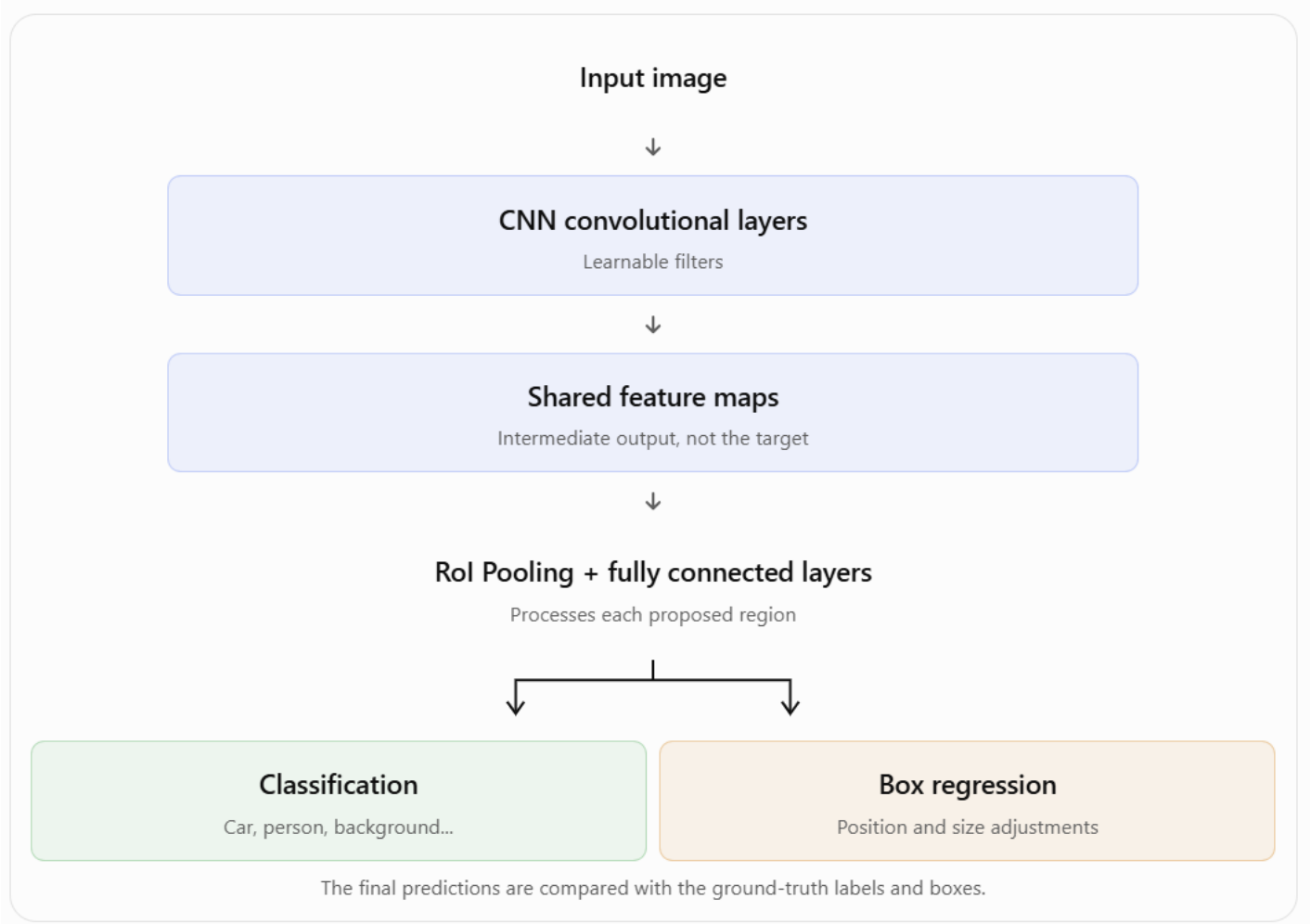
Then region proposals are mapped onto that shared feature map.

$$\text{Region Proposal} \rightarrow \text{Corresponding Feature-Map Region}$$

Key Improvement

All proposals share the same CNN computation.

Fast R-CNN Architecture



How Does Fast R-CNN Use the Feature Map?

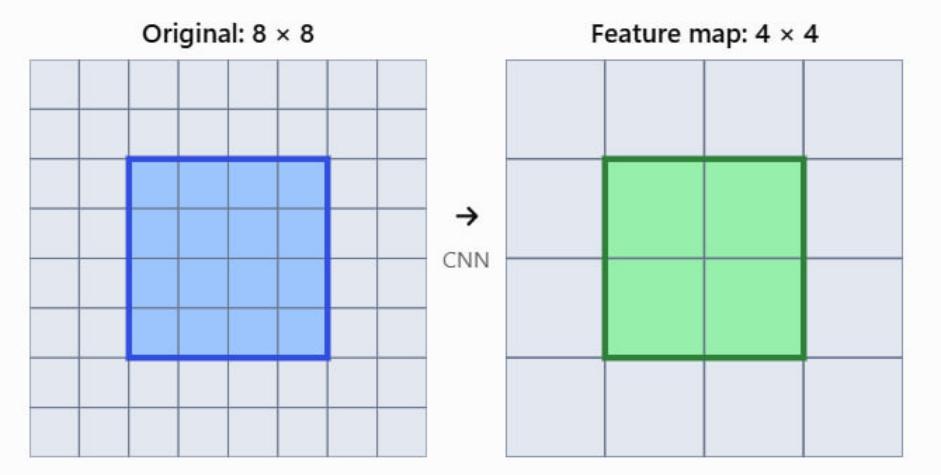
The CNN processes the entire image and produces a feature map.

$$\boxed{\text{Image} \rightarrow \text{CNN} \rightarrow \text{Feature Map}}$$

The "feature map" is actually a 3D feature tensor, which is a stack of many feature maps produced by many learned filters, all computed once for the whole image.

Each position in the feature map contains learned visual information from a region of the original image.

The original region proposal is projected onto the feature map.



Important

Fast R-CNN does not crop the original image and run a new CNN for every proposal.
It extracts the proposal's features from the already-computed shared feature map.

The Problem: Region Proposals Have Different Sizes

So far we know that the region proposals and their original sizes are first generated by Selective Search, then mapped onto the shared feature tensor.
But region proposals can have many different shapes and sizes.
For example:

$$50 \times 100, \qquad 120 \times 80, \qquad 200 \times 200$$

But the later classification layers require a fixed-size feature representation.

Fast R-CNN therefore introduced:

RoI Pooling

where RoI means:

Region of Interest

Goal

Convert features from differently sized proposals into the same fixed spatial size.

How RoI Pooling Works

Suppose a proposal corresponds to a rectangular region on the feature map.
RoI Pooling:

1. takes that feature-map region,
2. divides it into a fixed grid,
3. performs max pooling inside each grid cell,
4. produces a fixed-size output.

For example:

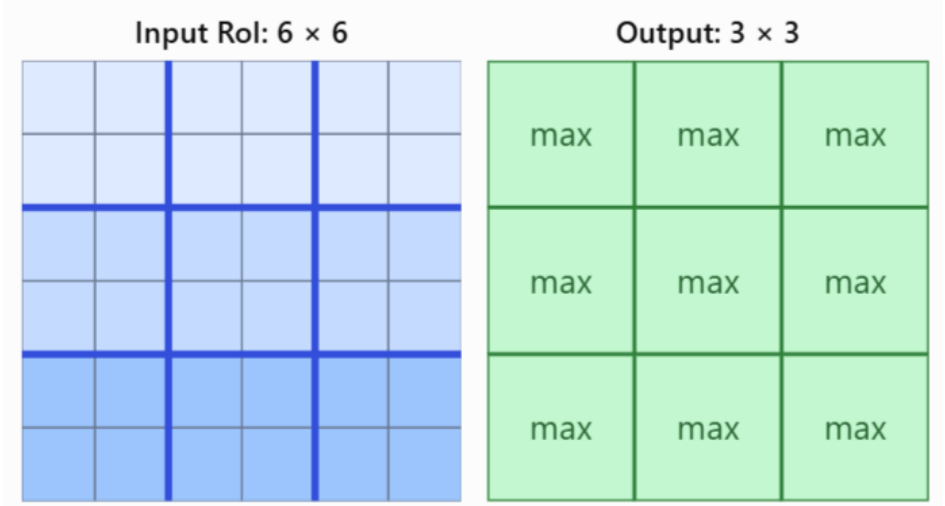
$$\text{Variable-size RoI} \rightarrow 3 \times 3 \text{ feature representation}$$

Why This Matters

Every proposal can now be passed through the same classification layers, regardless of its original size.

How does it divide a region into a fixed grid?

Suppose a region on one feature map has dimensions 6 × 6 and we want a 3 × 3 output:



The 6 × 6 region is divided into 9 bins. Each bin contains 2 × 2 values. RoI Pooling takes the maximum from each bin.

What Does Fast R-CNN Predict?

After RoI Pooling, the fixed-size feature representation is flattened into a 1D vector and then passed through fully connected layers for classification and bounding-box regression.
The network produces two outputs:

Class Scores

and

Bounding-Box Adjustments

The class prediction determines:

person, car, dog, background, etc.

The bounding-box regression predicts how to:

- shift the proposal,
- change its width,
- change its height.

How Is Fast R-CNN Trained?

Fast R-CNN trains classification and bounding-box regression together.

For each selected region proposal:

- assign an object class or background label,
- compare the predicted class with the true class,
- compare the predicted box adjustment with the target box.

The total loss contains two main parts:

$$\boxed{L = L_{\text{classification}} + \lambda L_{\text{box}}}$$

Improvement Over R-CNN

Classification and bounding-box refinement are trained within one network rather than as several completely separate stages.

Fast R-CNN Training and Inference

During training: The CNN processes the whole image once to generate shared feature maps. Multiple region proposals use those maps through RoI Pooling. Classification and bounding-box regression losses are used to update the CNN weights through backpropagation.

During inference: The CNN processes the whole image once to generate shared feature maps. Each proposal uses RoI Pooling, followed by classification and bounding-box refinement.

What Problem Did Fast R-CNN Still Have?

Fast R-CNN made feature extraction much faster.
However, it still depended on an external algorithm to generate region proposals.

The pipeline was still:

$$\boxed{\text{Image} \rightarrow \text{Selective Search} \rightarrow \text{Region Proposals}}$$

followed by:

$$\boxed{\text{CNN Feature Map} \rightarrow \text{RoI Pooling} \rightarrow \text{Classes + Boxes}}$$

Remaining Bottleneck

Region proposal generation was still outside the neural network and was not learned end-to-end.

Current Section

1 What object detection must do

2 The Original Region-Based Idea

3 R-CNN

4 Fast R-CNN

5 Faster R-CNN
- Lab

6 Mask R-CNN

Faster R-CNN: Learn the Region Proposals

Faster R-CNN solved the remaining proposal bottleneck.

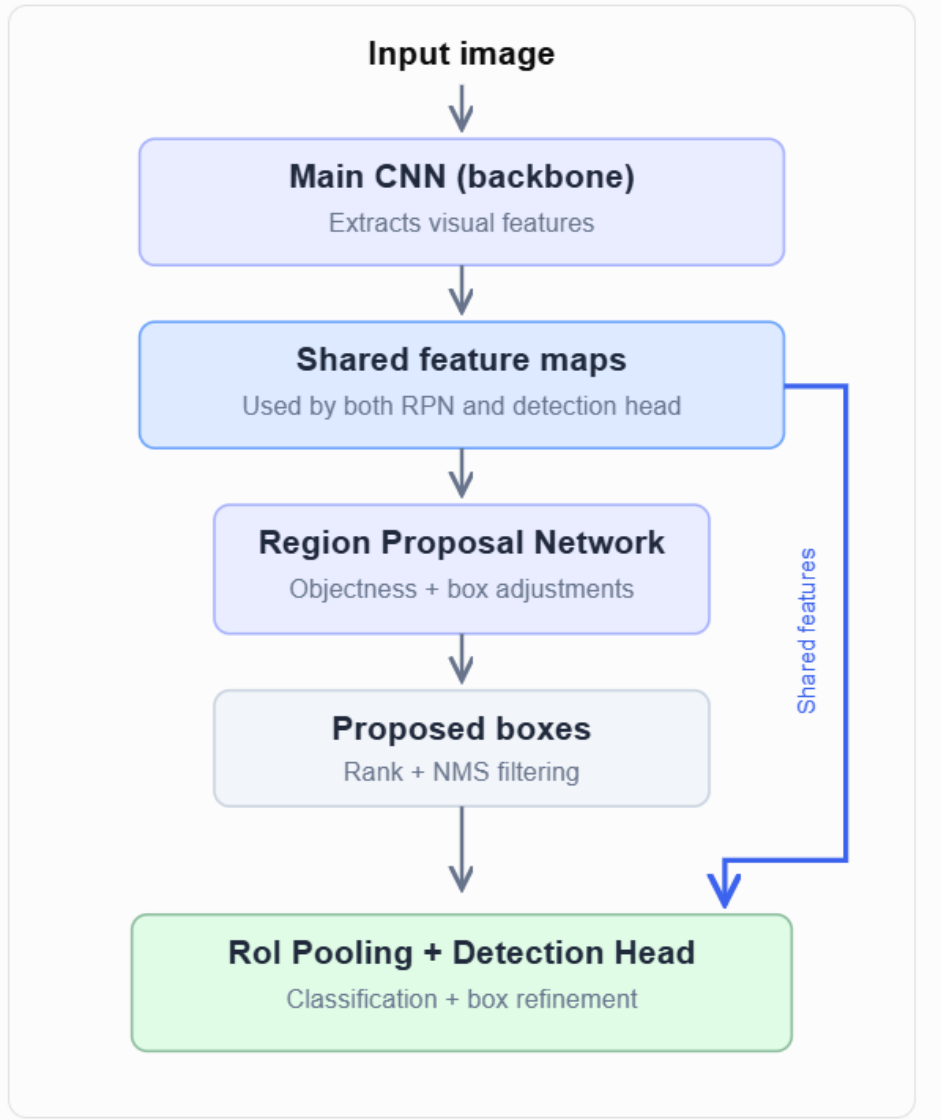
Instead of using Selective Search, it introduced a neural network called:

Region Proposal Network (RPN)

The RPN learns to generate candidate object regions directly from the CNN feature map.
The pipeline becomes:

$$\text{Image} \rightarrow \text{Backbone CNN} \rightarrow \text{Feature Map} \rightarrow \text{RPN} \rightarrow \text{Region Proposals}$$

Faster R-CNN Architectures



What Does the Region Proposal Network Do?

The RPN examines many spatial locations on the feature map.
At each location, it considers candidate boxes and predicts:

Objectness

Is this region likely to contain an object?
and

Bounding-Box Adjustment

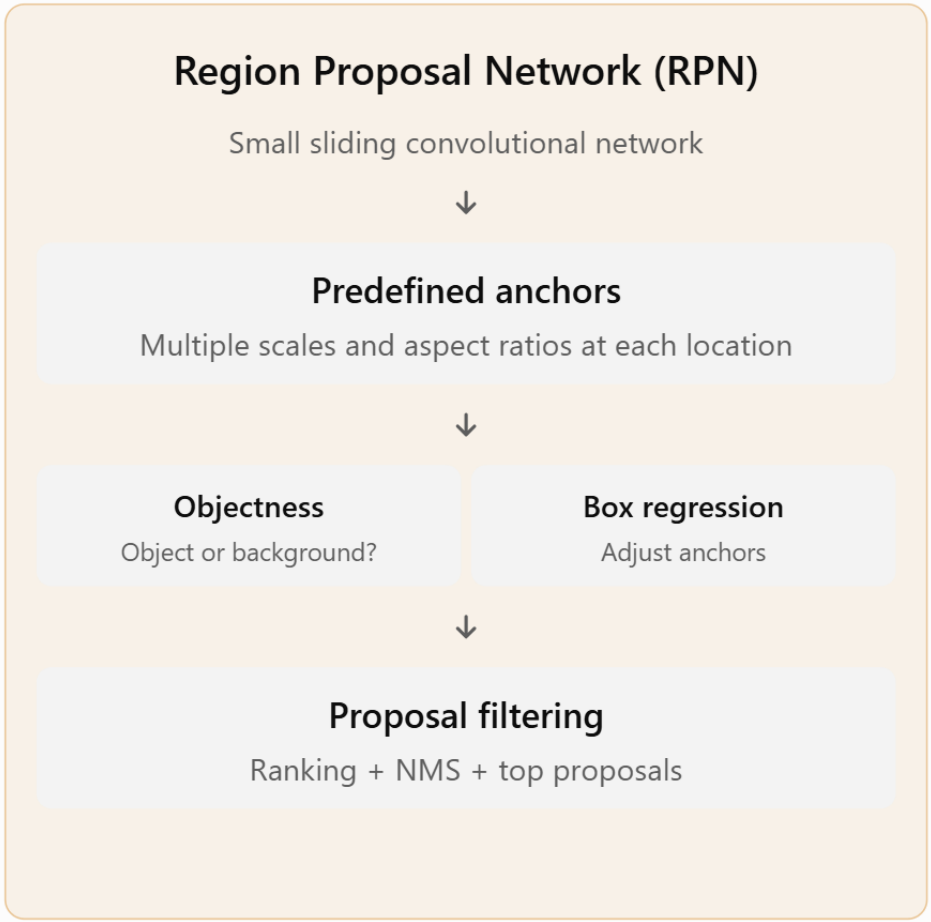
How should the candidate box move or resize?

RPN Question

The RPN does not need to determine the final object class.
It mainly asks:

"Is there probably an object here?"

RPN


Region Proposal Network (RPN)
Small sliding convolutional network
Predefined anchors
Multiple scales and aspect ratios at each location
Objectness
Object or background?
Box regression
Adjust anchors
Proposal filtering
Ranking + NMS + top proposals

How Does the RPN Generate Candidate Boxes?

Traditional Faster R-CNN uses predefined reference boxes called **anchor boxes**.

At many feature-map locations, several anchor shapes are considered.
For each anchor, the RPN predicts:

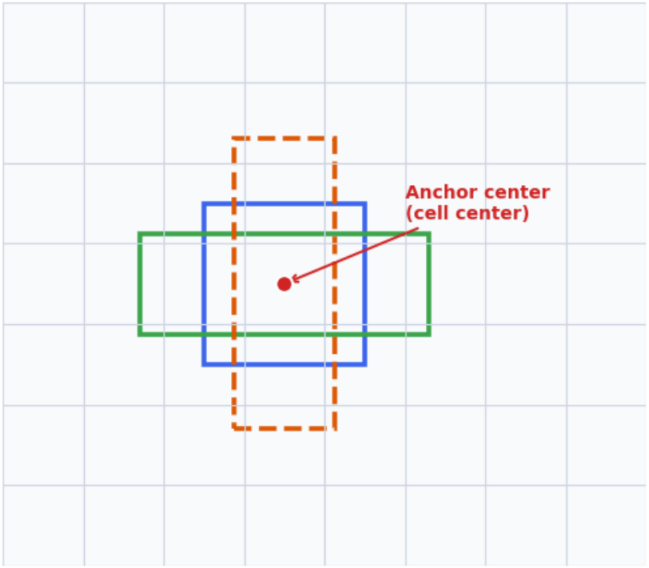
- object or background probability,
- box-coordinate adjustments.

$$\boxed{\text{Anchor} + \text{RPN Adjustment} \rightarrow \text{Region Proposal}}$$

For Now

Think of anchors as starting boxes.
We will study exactly how anchor boxes are created and trained in a later section.

Predefined Anchors



In the original Faster R-CNN, the RPN uses 9 predefined anchors at each feature-map cell, with combinations of three scales and three aspect ratios.
Each anchor is centered at the corresponding spatial location and has a predefined width and height.
The figure illustrates three anchor shapes sharing the same center. The original Faster R-CNN uses nine anchor shapes per location.

Faster R-CNN: Complete Pipeline

Faster R-CNN has two main detection stages.

Stage 1: Region Proposal Network

$$\text{Feature Map} \rightarrow \text{RPN} \rightarrow \text{Candidate Regions}$$

Stage 2: Detection Head

$$\text{Candidate Regions} \rightarrow \text{RoI Features} \rightarrow \text{Class + Refined Box}$$

Therefore:

$$\boxed{\text{Image} \rightarrow \text{Backbone} \rightarrow \text{RPN} \rightarrow \text{RoI Features} \rightarrow \text{Classes + Boxes}}$$

Why Is Faster R-CNN Faster?

The key improvement is shared computation.

The backbone CNN computes the feature map only once:

$$\text{Image} \rightarrow \boxed{\text{Shared Feature Map}}$$

Both components use that same feature map:

$$\text{Shared Features} \rightarrow \begin{cases} \text{RPN} \\ \text{Detection Head} \end{cases}$$

Major Improvement

Region proposals are now generated by a learned neural network instead of an external Selective Search algorithm.

How Is Faster R-CNN Trained?

Faster R-CNN trains two components using ground-truth object classes and bounding boxes.

1. **Region Proposal Network (RPN)**
   - Predicts objectness (object vs. background).
   - Learns bounding-box adjustments to generate proposals.
2. **Second-Stage Detection Head**
   - Predicts the object class.
   - Refines the proposed bounding boxes.

The total training loss combines classification and bounding-box regression losses from both components:

$$L_{\text{total}} = L_{\text{RPN-object}} + L_{\text{RPN-box}} + L_{\text{class}} + L_{\text{final-box}}$$

Key Idea

Backpropagation updates the RPN, detection head, and shared CNN backbone to improve object classification and localization.

Faster R-CNN During Inference

For a new image:

1. the backbone creates shared feature maps,
2. the RPN scores and adjusts anchors,
3. candidate proposals are filtered,
4. NMS removes redundant proposals,
5. RoI features are extracted,
6. the detection head predicts classes and refined boxes,
7. final confidence filtering and NMS produce detections.

$$\text{Image} \rightarrow \text{RPN} \rightarrow \text{Proposals} \rightarrow \text{RoI Features} \rightarrow \text{Classes + Boxes} \rightarrow \text{NMS}$$

Fast R-CNN vs. Faster R-CNN

Both models share CNN features across proposals.
The main difference is how proposals are generated.

Aspect	Fast R-CNN	Faster R-CNN
Feature extraction	Shared CNN	Shared CNN
Region proposals	Selective Search	Learned RPN
Proposal learning	No	Yes
Final classification	Learned	Learned
Box refinement	Learned	Learned

Major Innovation

Faster R-CNN brings region proposal generation into the neural network.

Current Subsection

1 What object detection must do

2 The Original Region-Based Idea

3 R-CNN

4 Fast R-CNN

5 Faster R-CNN
- Lab

6 Mask R-CNN

Imports

```
!pip -q install tensorflow-hub pillow matplotlib requests
```

```
import tensorflow as tf
import tensorflow_hub as hub
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as patches
import requests
from PIL import Image
from io import BytesIO

print('TensorFlow version:', tf.__version__)
```

```
/usr/local/lib/python3.13/dist-packages/tensorflow_hub/__init__.py:61:
  from pkg_resources import parse_version
TensorFlow version: 2.21.0
```

Loading an Image

```python
# One COCO image
url = 'https://huggingface.co/datasets/hf-internal-testing/fixtures-coco/resolve/main/val2017/000000039769.jpg'
response = requests.get(url, timeout=60)
response.raise_for_status()
image = np.array(Image.open(BytesIO(response.content)).convert('RGB'))

plt.figure(figsize=(8, 5))
plt.imshow(image)
plt.axis('off')
plt.title('Original image')
plt.show()
```



COCO Dataset: Common Objects in Context

What is COCO?

- COCO stands for **Common Objects in Context**.
- A widely used dataset for training and evaluating computer vision models.
- Contains over 200,000 labeled images, including about 118,000 training images in COCO 2017.
- Includes 80 object categories, such as people, cats, dogs, cars, and bicycles.

What information does COCO provide?

- **Class labels:** What objects are present?
- **Bounding boxes:** Where are the objects?
- **Segmentation masks:** Which pixels belong to each object?

Connection to Faster R-CNN:

COCO is the **dataset**, while Faster R-CNN is the **object detection architecture** trained on it.

Faster R-CNN: Example Using COCO

- We use image `000000039769.jpg` from the COCO 2017 validation set.
- The image contains two cats on a couch.
- A pretrained Faster R-CNN model detects objects and predicts their bounding boxes.

How Faster R-CNN processes the image:

1. Backbone CNN extracts visual features.
2. Region Proposal Network (RPN) identifies candidate object regions.
3. RoI features are used to classify objects and refine bounding boxes.

Use pretrained Faster R-CNN on the Image

```
# Load the pretrained model from TensorFlow Hub
model_url = 'https://tfhub.dev/tensorflow/faster_rcnn/resnet50_v1_640x640/1'
model = hub.load(model_url)

# Faster R-CNN expects a batch of uint8 images
predictions = model(tf.convert_to_tensor(image[None, ...], dtype=tf.uint8))
print('Outputs:', list(predictions.keys()))
```

```
Outputs: ['raw_detection_boxes', 'detection_anchor_indices', 'num_detections', 'detection_scores', 'raw_detection_scores',
```

Pretrained Faster R-CNN: Model Outputs

Model: Faster R-CNN with a ResNet-50 backbone, pretrained on COCO, detects and localizes objects using a Region Proposal Network (RPN).

Model Outputs:

- `raw_detection_boxes`: Predicted boxes before final post-processing.
- `detection_anchor_indices`: Anchor/proposal indices associated with detections.
- `num_detections`: Number of valid detections.
- `raw_detection_scores`: Class scores for candidate bounding boxes before final post-processing.
- `detection_scores`: Confidence scores for the selected detections after filtering and Non-Maximum Suppression (NMS).
- `detection_classes`: Predicted object class IDs.
- `detection_multiclass_scores`: Class scores for each detection.
- `detection_boxes`: Final bounding boxes, typically normalized coordinates.

Display All Bounding Boxes

We are using a confidence threshold of 0.5:

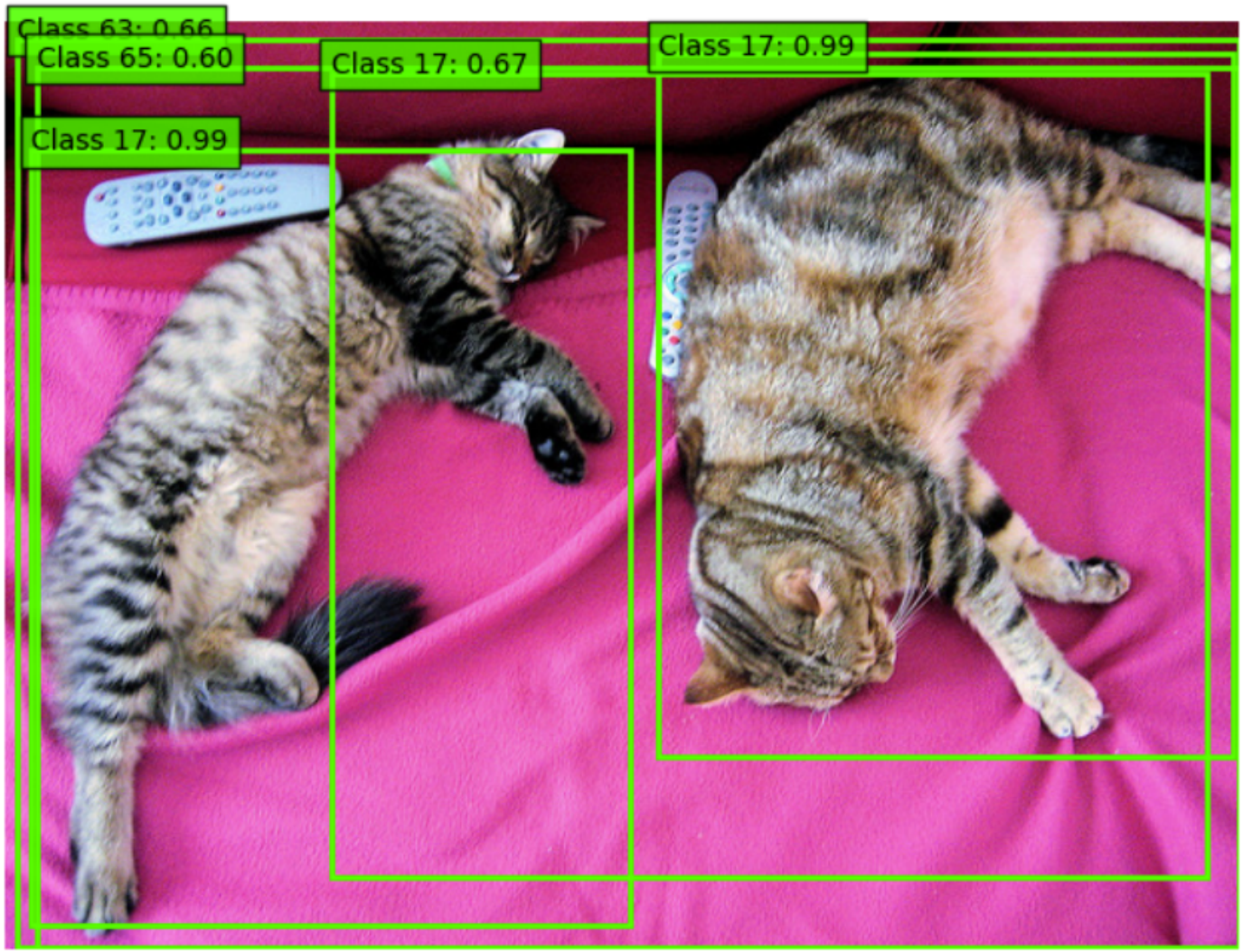
```
def show_boxes(image, predictions, threshold=0.5):
    boxes = predictions['detection_boxes'][0].numpy()
    scores = predictions['detection_scores'][0].numpy()
    classes = predictions['detection_classes'][0].numpy().astype(int)
    height, width = image.shape[:2]
    fig, ax = plt.subplots(figsize=(9, 6))
    ax.imshow(image)

    for box, score, label in zip(boxes, scores, classes):
        if score < threshold: continue
        top, left, bottom, right = box
        x, y = left * width, top * height
        w = (right - left) * width
        h = (bottom - top) * height
        ax.add_patch(patches.Rectangle((x, y), w, h, fill=False, edgecolor='lime', linewidth=2))
        ax.text(x, y, f'Class {label}: {score:.2f}', color='black', bbox=dict(facecolor='lime', alpha=0.8))

    ax.axis('off')
    plt.show()

show_boxes(image, predictions, threshold=0.5)
```

Result


Class 63: 0.66
Class 65: 0.60
Class 17: 0.67
Class 17: 0.99
Class 17: 0.99

Loading COCO Class Labels

Downloading the COCO label map and creates a dictionary that converts numeric class IDs into object names, such as 17 → cat.

```
import requests, re

url = "https://raw.githubusercontent.com/tensorflow/models/master/research/object_detection/data/mscoco_label_map.pbtxt"
text = requests.get(url).text
coco_names = {int(i): name for i, name in re.findall(r'id:\s*(\d+)\s+display_name:\s*"([^"]+)"', text)}
print(coco_names.get(17))
```

```
cat
```

Using COCO Class Labels

We modify the show_boxes() function to use coco_names.get() to display object names instead of numeric class IDs, along with their confidence scores.

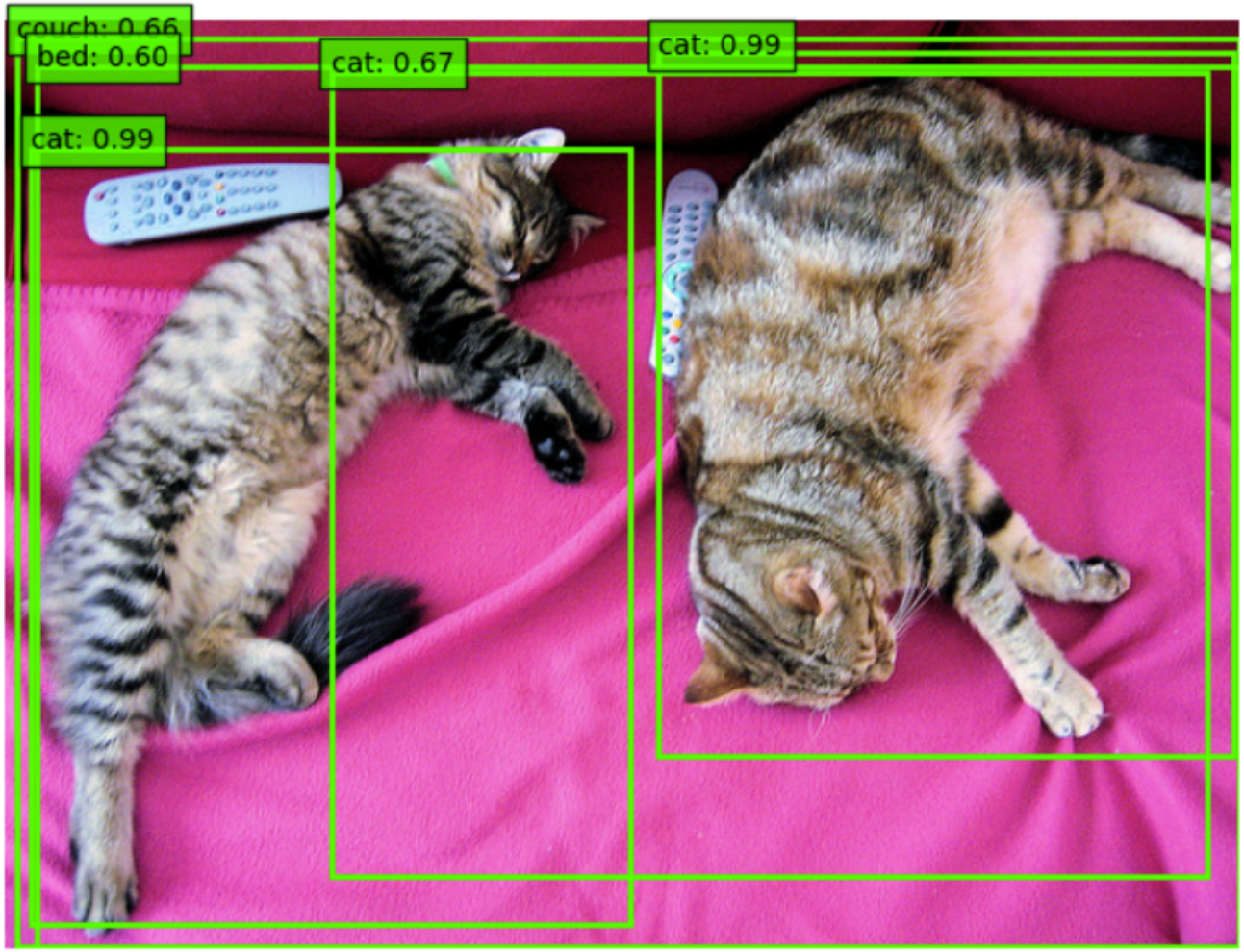
```python
def show_boxes(image, predictions, threshold=0.5):
    boxes = predictions['detection_boxes'][0].numpy()
    scores = predictions['detection_scores'][0].numpy()
    classes = predictions['detection_classes'][0].numpy().astype(int)
    height, width = image.shape[:2]
    fig, ax = plt.subplots(figsize=(9, 6))
    ax.imshow(image)

    for box, score, label in zip(boxes, scores, classes):
        if score < threshold: continue
        top, left, bottom, right = box
        x, y = left * width, top * height
        w, h = (right - left) * width, (bottom - top) * height
        ax.add_patch(patches.Rectangle((x, y), w, h, fill=False, edgecolor='lime', linewidth=2))
        name = coco_names.get(label, f'Class {label}')
        ax.text(x, y, f'{name}: {score:.2f}', color='black', bbox=dict(facecolor='lime', alpha=0.8))

    ax.axis('off')
    plt.show()

show_boxes(image, predictions, threshold=0.5)
```

Result



Faster R-CNN detects objects with confidence scores above 0.5, identifying two cats with high confidence while also predicting overlapping boxes for the couch and bed.

Filtering by Object Class

We add the only_class parameter to display only cats (class 17) with confidence scores of at least 0.5, excluding all other detected objects.

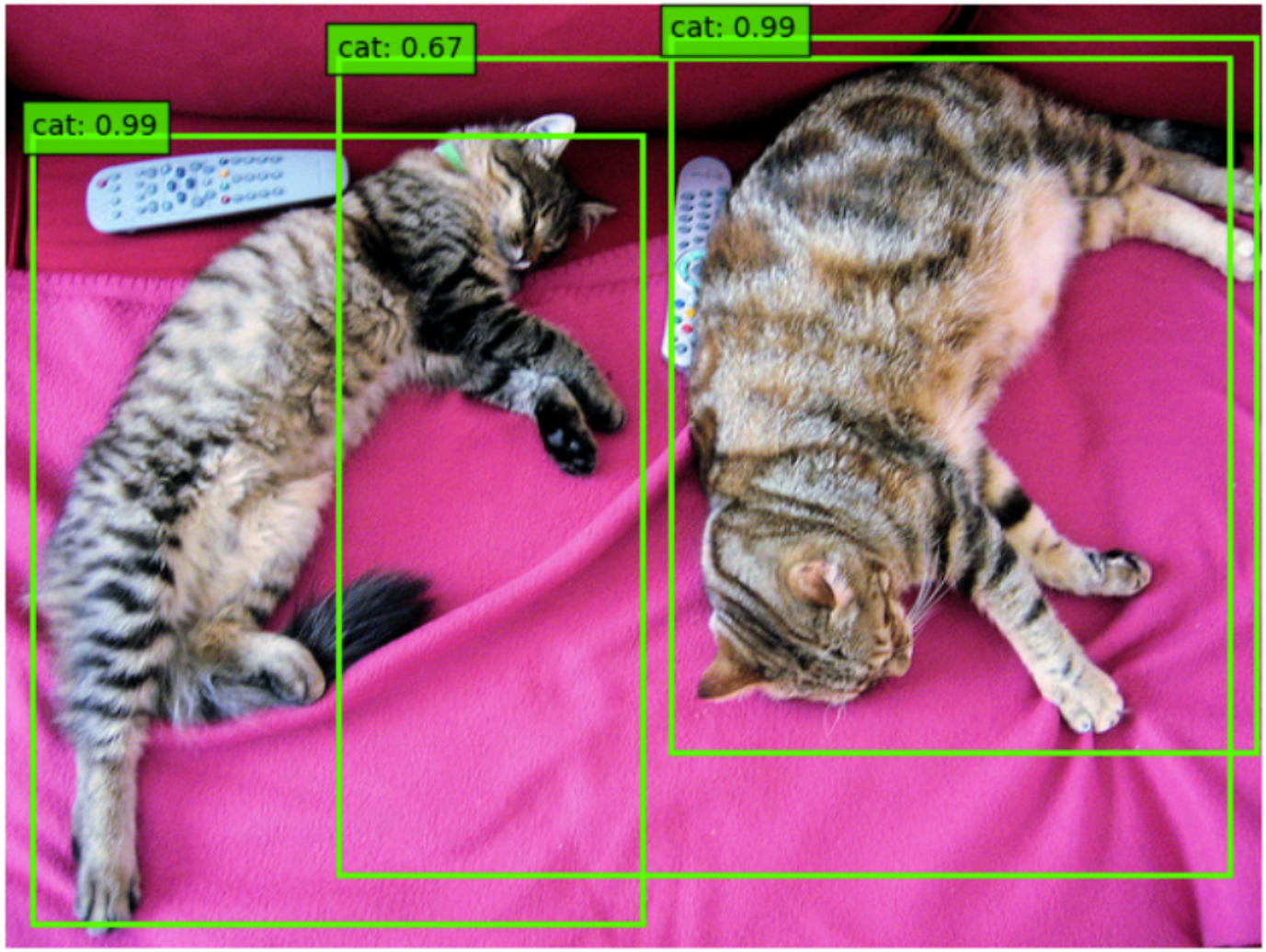
```
def show_boxes(image, predictions, threshold=0.5, only_class=None):
    boxes = predictions['detection_boxes'][0].numpy()
    scores = predictions['detection_scores'][0].numpy()
    classes = predictions['detection_classes'][0].numpy().astype(int)
    height, width = image.shape[:2]
    fig, ax = plt.subplots(figsize=(9, 6))
    ax.imshow(image)

    for box, score, label in zip(boxes, scores, classes):
        if score < threshold or (only_class is not None and label != only_class): continue
        top, left, bottom, right = box
        x, y = left * width, top * height
        w, h = (right - left) * width, (bottom - top) * height
        ax.add_patch(patches.Rectangle((x, y), w, h, fill=False, edgecolor='lime', linewidth=2))
        name = coco_names.get(label, f'Class {label}')
        ax.text(x, y, f'{name}: {score:.2f}', color='black', bbox=dict(facecolor='lime', alpha=0.8))

    ax.axis('off')
    plt.show()

show_boxes(image, predictions, threshold=0.5, only_class=17)
```

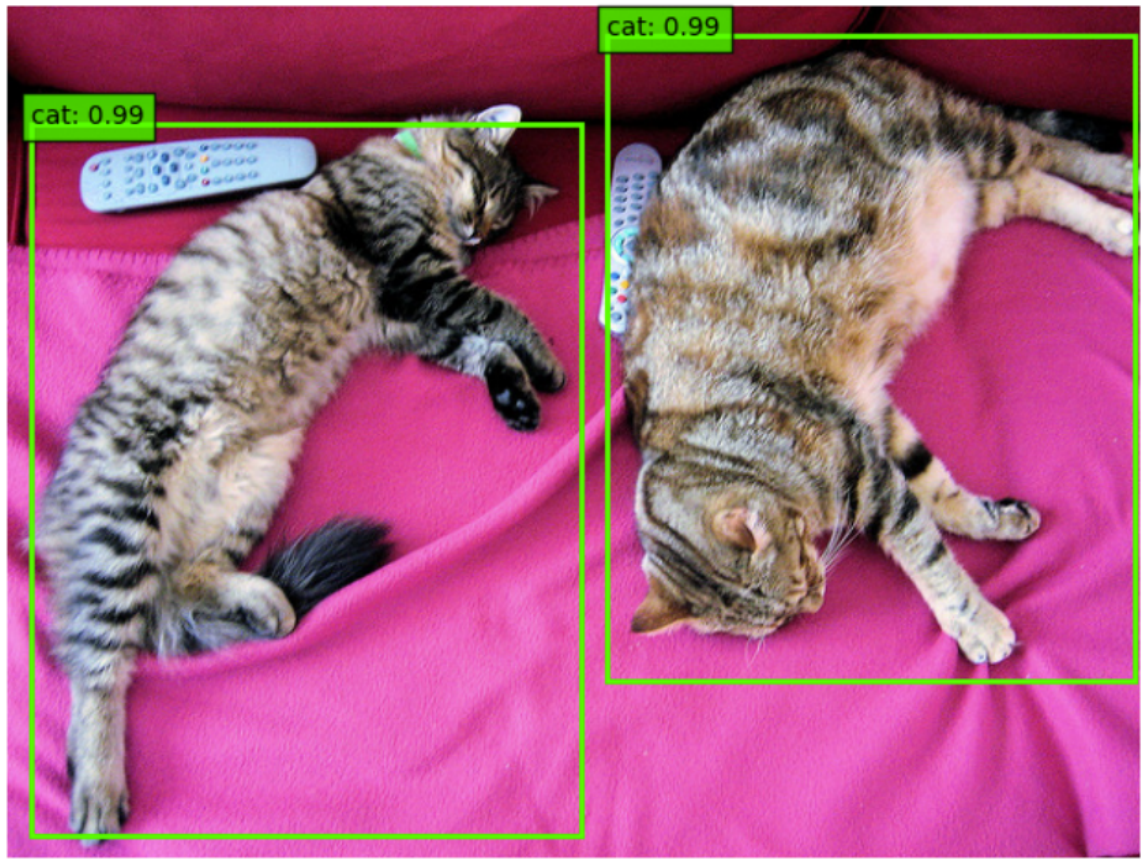
Result



Increasing the Confidence Threshold

Setting threshold=0.9 displays only cats with confidence scores of at least 90%, removing lower-confidence detections and retaining the two cats with scores of 0.99.

```
show_boxes(image, predictions, threshold=0.9, only_class=17)
```



Comparing TensorFlow and PyTorch Faster R-CNN

```python
import torch
from torchvision.models.detection import fasterrcnn_resnet50_fpn, FasterRCNN_ResNet50_FPN_Weights
from PIL import Image

weights = FasterRCNN_ResNet50_FPN_Weights.DEFAULT
torch_model = fasterrcnn_resnet50_fpn(weights=weights).eval()
torch_image = Image.fromarray(image)
torch_input = weights.transforms()(torch_image)

with torch.inference_mode():
    torch_predictions = torch_model([torch_input])[0]

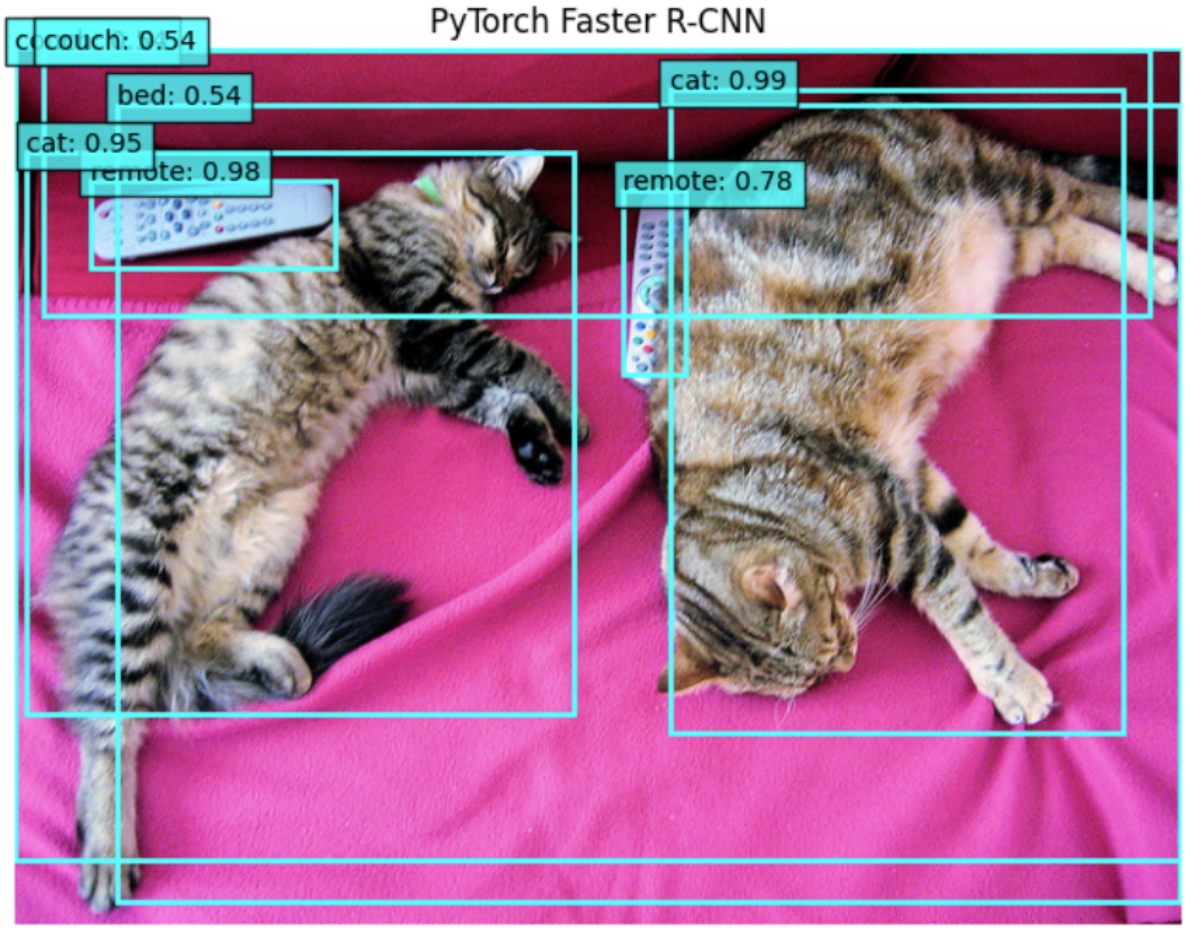
fig, ax = plt.subplots(figsize=(9, 6))
ax.imshow(image)

for box, score, label in zip(torch_predictions['boxes'], torch_predictions['scores'], torch_predictions['labels']):
    if score < 0.5: continue
    x1, y1, x2, y2 = box.tolist()
    name = weights.meta['categories'][label.item()]
    ax.add_patch(patches.Rectangle((x1, y1), x2-x1, y2-y1, fill=False, edgecolor='cyan', linewidth=2))
    ax.text(x1, y1, f'{name}: {score:.2f}', color='black', bbox=dict(facecolor='cyan', alpha=0.8))

ax.set_title('PyTorch Faster R-CNN')
ax.axis('off')
plt.show()
```

Result

We apply a COCO-pretrained Faster R-CNN model from PyTorch to the same image using a confidence threshold of 0.5. Although both models use the Faster R-CNN architecture with a ResNet-50 backbone, they may detect different objects, such as cats and remote controls, due to differences in training, preprocessing, and post-processing.



Another Example

We evaluate the pretrained TensorFlow Faster R-CNN model on three images from the Penn-Fudan Pedestrian dataset, which contains images of people and their segmentation masks.

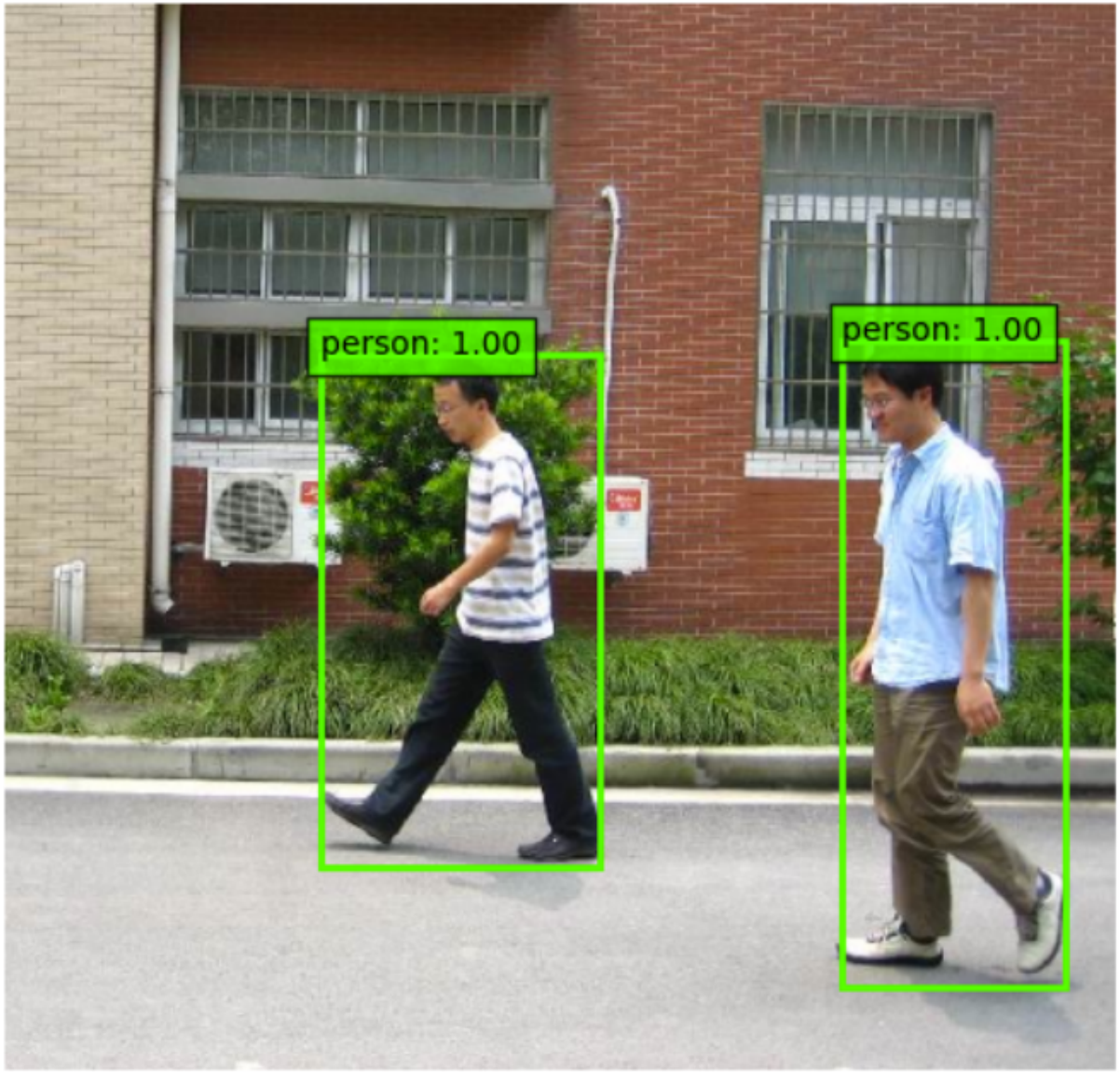
```
from pathlib import Path
from io import BytesIO
import zipfile

folder = Path('PennFudanPed')

if not folder.exists():
    url = 'https://www.cis.upenn.edu/~jshi/ped_html/PennFudanPed.zip'
    with zipfile.ZipFile(BytesIO(requests.get(url).content)) as z: z.extractall('.')

for path in sorted((folder / 'PNGImages').glob('*.png'))[:3]:
    photo = np.array(Image.open(path).convert('RGB'))
    mask = np.array(Image.open(folder / 'PedMasks' / f'{path.stem}_mask.png'))
    actual = len(np.unique(mask)) - 1
    result = model(tf.convert_to_tensor(photo[None, ...], dtype=tf.uint8))
    scores = result['detection_scores'][0].numpy()
    classes = result['detection_classes'][0].numpy().astype(int)
    predicted = np.sum((classes == 1) & (scores >= 0.5))
    print(f'{path.name}: Actual = {actual}, Predicted = {predicted}')
    show_boxes(photo, result, threshold=0.5, only_class=1)
```

Result



Current Section

1 What object detection must do

2 The Original Region-Based Idea

3 R-CNN

4 Fast R-CNN

5 Faster R-CNN

6 Mask R-CNN
- Lab

Mask R-CNN: Adding Instance Segmentation

Mask R-CNN extends Faster R-CNN.
It keeps the same basic detection structure:

$$\text{RPN} \rightarrow \text{Region Classification} \rightarrow \text{Bounding-Box Refinement}$$

but adds another output:

$$\boxed{\text{Object Mask}}$$

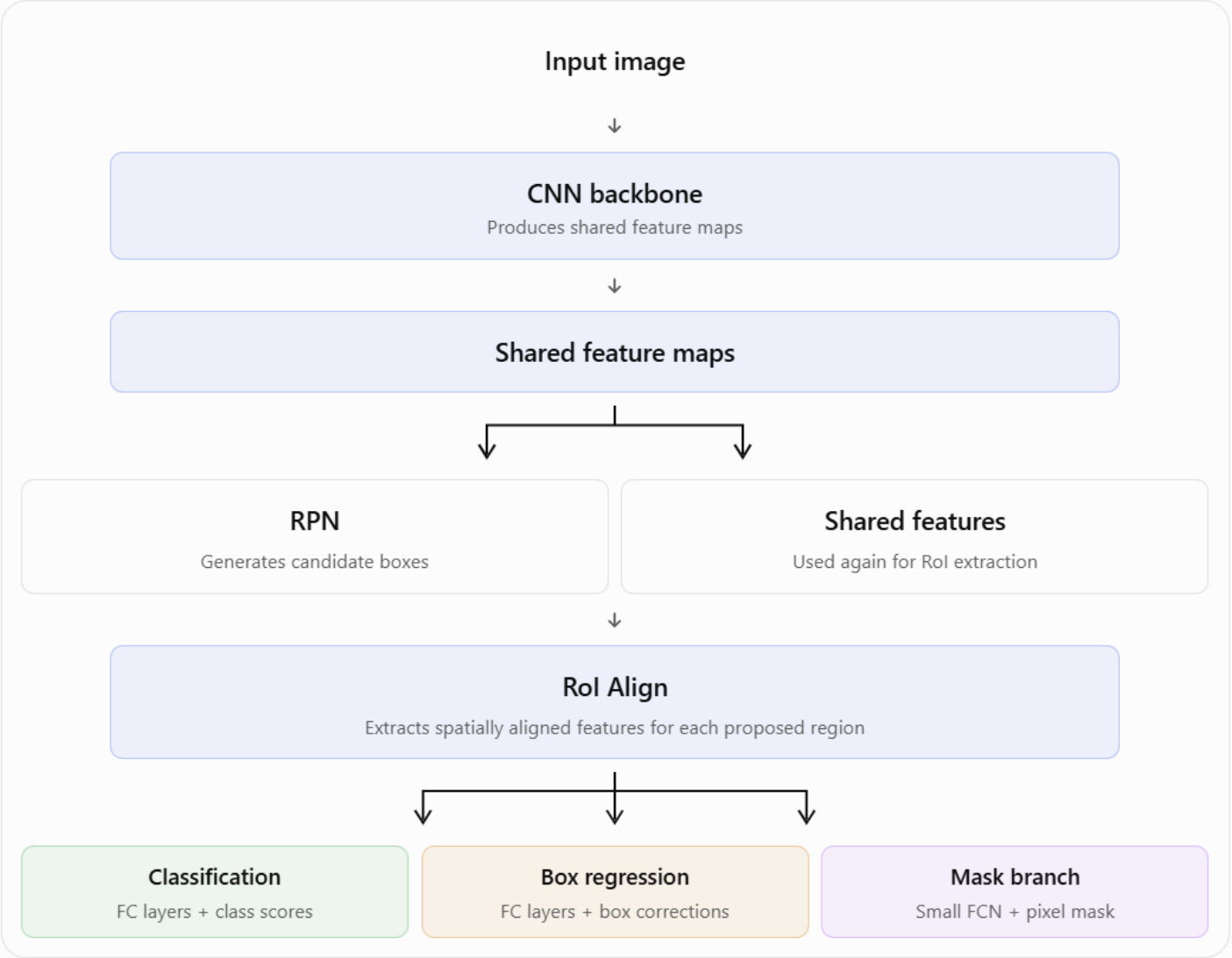
For every detected object, Mask R-CNN can predict:

- class label,
- bounding box,
- pixel-level segmentation mask.

Difference

Faster R-CNN tells us where the object box is.
Mask R-CNN can also identify which pixels belong to that object.

Mask R-CNN Architecture



Why Did Mask R-CNN Introduce RoI Align?

RoI Pooling rounds region coordinates when mapping them onto the feature map. This small misalignment is often acceptable for bounding boxes, but it can hurt pixel-level segmentation.

Mask R-CNN therefore uses:

RoI Align

RoI Align preserves more precise spatial alignment by avoiding coarse coordinate rounding to integers. Instead, it uses bilinear interpolation to estimate feature values at precise fractional locations.

Why This Matters

Segmentation masks require more precise correspondence between the original object and its feature-map representation.

Evolution of the R-CNN Family

Model	Region Proposals	CNN Processing	Main Improvement
R-CNN	Selective Search	CNN per region	Introduced CNN-based
Fast R-CNN	Selective Search	CNN once per image	Shared feature map
Faster R-CNN	Learned RPN	CNN once per image	Learned proposals
Mask R-CNN	Learned RPN	CNN once per image	Adds object masks

R-CNN $\rightarrow$ Fast R-CNN $\rightarrow$ Faster R-CNN $\rightarrow$ Mask R-CNN

What Changed Across the R-CNN Family?

The basic region-based idea stayed the same:

Find Regions $\rightarrow$ Classify and Refine Them

What changed was **how efficiently the regions and features were produced**.

R-CNN: CNN separately for every region

Fast R-CNN: shared feature map

Faster R-CNN: learned region proposals

Mask R-CNN: adds pixel-level masks

What Type of Detector Is the R-CNN Family?

R-CNN-style detectors separate detection into two conceptual stages:

Stage 1: Find candidate regions

Stage 2: Classify and refine them

This family is therefore associated with:

Two-Stage Object Detection

Next Question

Do all object detectors need a separate region-proposal stage?

The answer leads to:

One-Stage vs. Two-Stage Detectors

Current Subsection

1 What object detection must do

2 The Original Region-Based Idea

3 R-CNN

4 Fast R-CNN

5 Faster R-CNN

6 Mask R-CNN
- Lab

Imports

```
import tensorflow as tf
import tensorflow_hub as hub
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as patches

url = 'https://tfhub.dev/tensorflow/mask_rcnn/inception_resnet_v2_1024x1024/1'
mask_model = hub.load(url)
```

Predictions

```
mask_predictions = mask_model(tf.convert_to_tensor(image[None, ...], dtype=tf.uint8))
print(mask_predictions.keys())
```

```
dict_keys(['anchors', 'num_detections', 'proposal_boxes', 'detection_masks', 'detection_scores',
```

Displaying Results

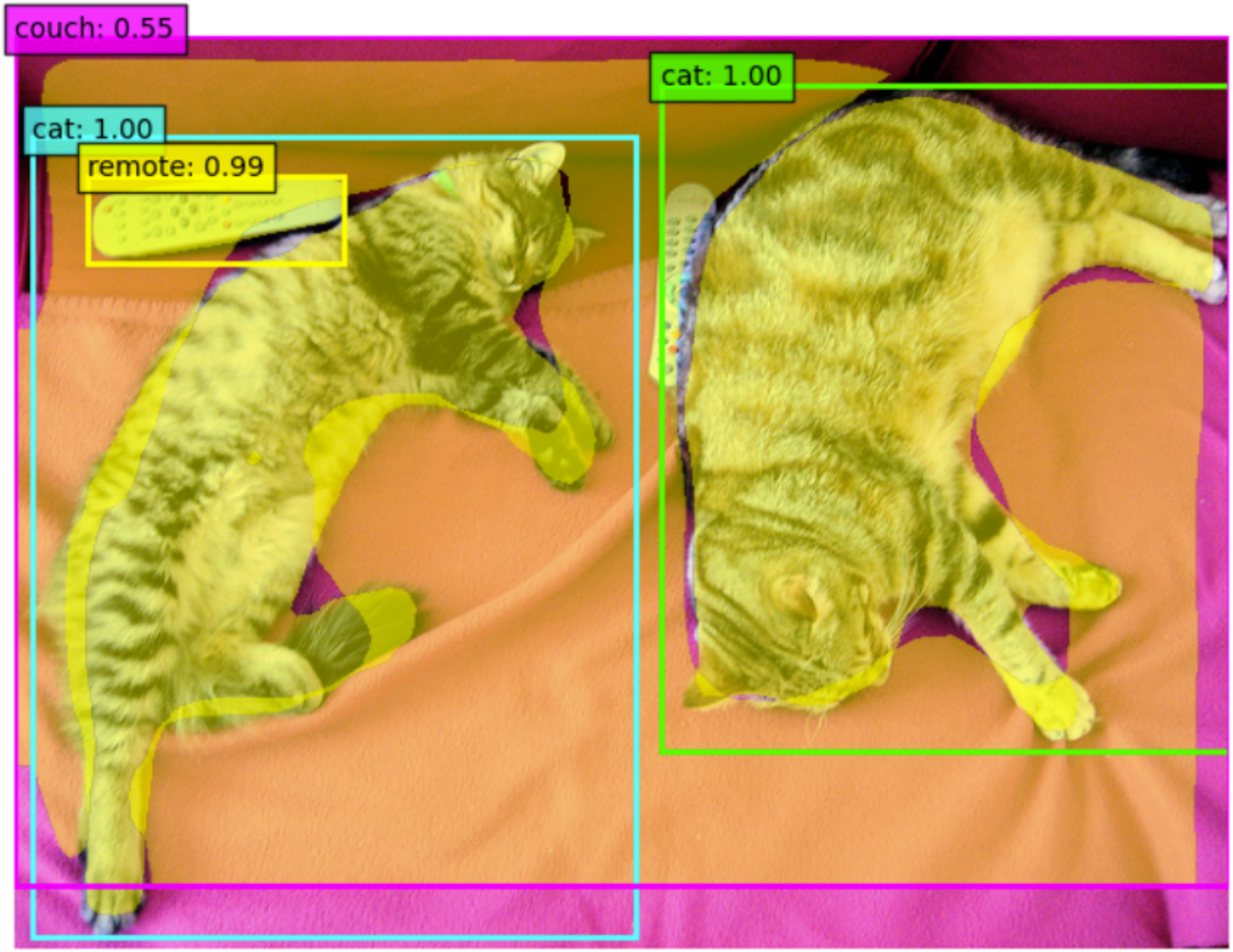
```python
def show_masks(image, predictions, threshold=0.5, only_class=None):
    boxes = predictions['detection_boxes'][0].numpy()
    scores = predictions['detection_scores'][0].numpy()
    classes = predictions['detection_classes'][0].numpy().astype(int)
    masks = predictions['detection_masks'][0].numpy()
    height, width = image.shape[:2]
    fig, ax = plt.subplots(figsize=(9, 6))
    ax.imshow(image)
    colors = ['lime', 'cyan', 'yellow', 'magenta']
    count = 0

    for box, score, label, mask in zip(boxes, scores, classes, masks):
        if score < threshold or (only_class is not None and label != only_class): continue
        top, left, bottom, right = box
        x1, y1, x2, y2 = int(left * width), int(top * height), int(right * width), int(bottom * height)
        x1, y1, x2, y2 = max(0, x1), max(0, y1), min(width, x2), min(height, y2)
        if x2 <= x1 or y2 <= y1: continue
        resized_mask = tf.image.resize(mask[..., None], (y2-y1, x2-x1)).numpy()[:, :, 0] > 0.5
        full_mask = np.zeros((height, width), dtype=bool)
        full_mask[y1:y2, x1:x2] = resized_mask
        ax.imshow(np.ma.masked_where(~full_mask, full_mask), cmap='spring', alpha=0.45, vmin=0, vmax=1)
        ax.add_patch(patches.Rectangle((x1, y1), x2-x1, y2-y1, fill=False, edgecolor=colors[count % 4], linewidth=2))
        name = coco_names.get(label, f'Class {label}')
        ax.text(x1, y1, f'{name}: {score:.2f}', color='black', bbox=dict(facecolor=colors[count % 4], alpha=0.8))
        count += 1

    ax.axis('off')
    plt.show()
    print(f'Objects detected: {count}')

show_masks(image, mask_predictions, threshold=0.5, only_class=None)
```

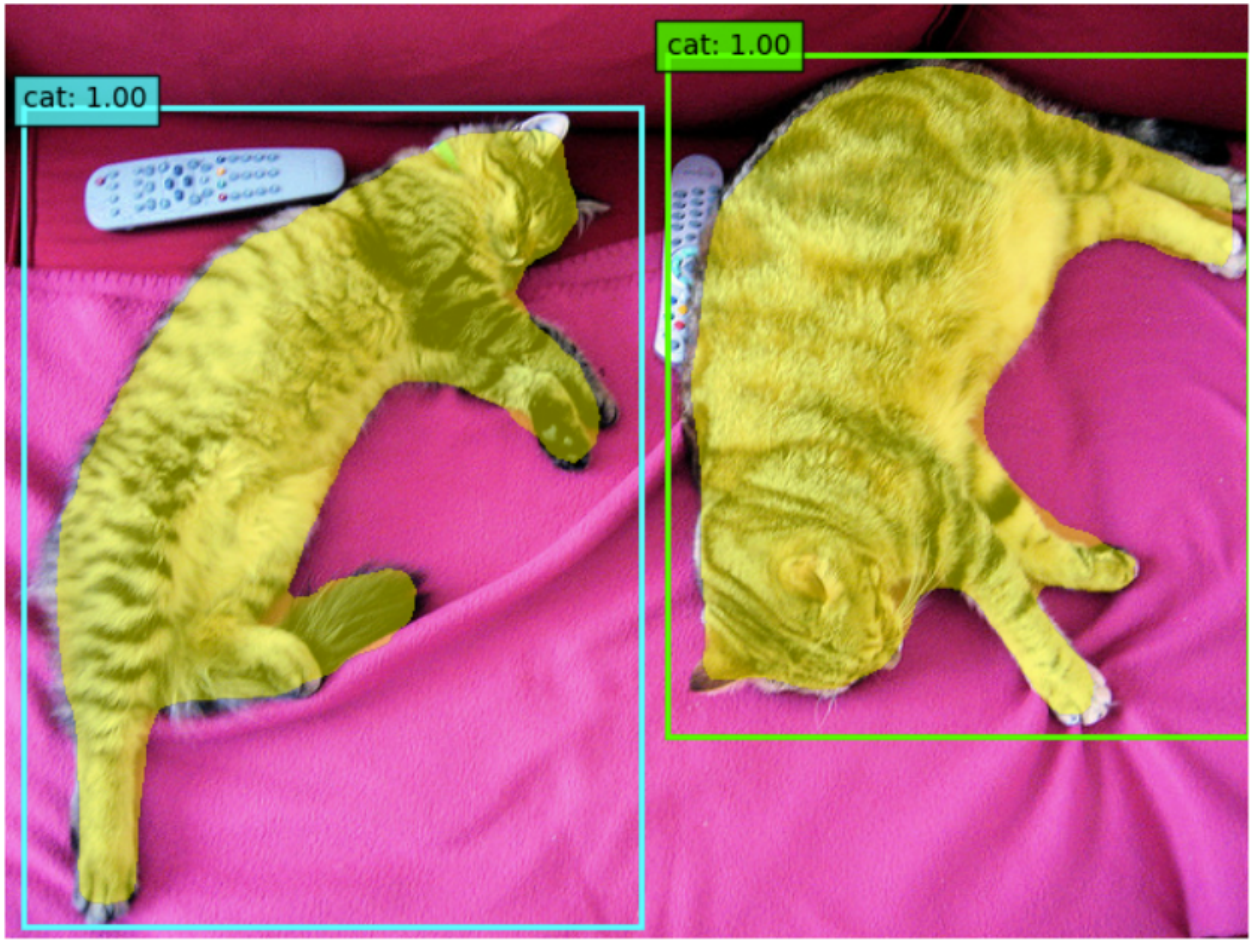
Results




```
Objects detected: 4
```

Displaying Only Cats

```
show_masks(image, mask_predictions, threshold=0.5, only_class=17)
```




```
Objects detected: 2
```

Congratulations!

Now you know about R-CNN, Fast R-CNN, and Faster R-CNN!

